

Day 3 · Topic 4: Model Formats Deep-Dive, Hardware Selection & CPU vs GPU Trade-offs

Week 1 — Local Inferencing Foundations

Goal: Convert a HuggingFace model to GGUF with llama.cpp, load SafeTensors and inspect file structure, calculate VRAM requirements for any model, understand FP32/FP16/INT8 memory footprints, and benchmark the same model on CPU vs GPU to quantify the trade-offs.

PART A — GGUF & SafeTensors File Internals

SafeTensors File Layout

A SafeTensors file is a flat binary format with two sections: a fixed-length 8-byte header storing the metadata size as a little-endian uint64, followed by a UTF-8 JSON blob containing tensor names, dtypes, shapes, and byte offsets. Immediately after the JSON come the raw tensor bytes, tightly packed. This layout enables **memory-mapped (mmap) zero-copy loading** — the OS maps the file into virtual address space and tensor data is only read from disk on first access, not eagerly loaded into RAM. This makes cold-start significantly faster for large sharded models.

```
# SafeTensors file structure (conceptual)
#
# Offset 0      : [8 bytes] metadata_length (little-endian uint64)
# Offset 8      : [metadata_length bytes] JSON header
#
#               {
#                 "__metadata__": {"format": "pt"},
#                 "model.embed_tokens.weight": {
#                   "dtype": "F16",
#                   "shape": [32000, 4096],
#                   "data_offsets": [0, 262144000]
#                 },
#                 ...
#               }
# Offset 8+N    : raw tensor bytes (no padding, no alignment)
```

```
import struct, json
```

```
def inspect_safetensors(path: str):
    with open(path, "rb") as f:
        meta_len = struct.unpack("<Q", f.read(8))[0]
        header = json.loads(f.read(meta_len))

    tensors = {k: v for k, v in header.items() if k != "__metadata__"}
    total_params = 0
    print(f"{'Tensor':<45} {'dtype':<6} {'shape'}")
    print("-" * 80)
    for name, info in list(tensors.items())[:20]: # show first 20
```

```

        shape = info["shape"]
        numel = 1
        for d in shape: numel *= d
        total_params += numel
        print(f"{name:<45} {info['dtype']:<6} {shape}")
    print(f"\n... ({len(tensors)} tensors total)")
    print(f"Approx total params: {total_params:,}")

# Usage:
# inspect_safetensors("~/cache/huggingface/hub/models--gpt2/model.safetensors")

```

Loading SafeTensors with HuggingFace

```

from transformers import AutoModelForCausalLM, AutoTokenizer
import torch, os, time

MODEL_ID = "gpt2" # ~500 MB – swap to "meta-llama/Meta-Llama-3-8B" for 8B

# Force SafeTensors format (default on modern HF Hub models)
t0 = time.perf_counter()
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    torch_dtype=torch.float16, # load in FP16 to halve RAM
    use_safetensors=True,     # explicitly request SafeTensors
    device_map="auto",        # auto-place on GPU if available, else CPU
    low_cpu_mem_usage=True,   # stream weights in – don't double-buffer
)
load_time = time.perf_counter() - t0

# Inspect loaded model
total_params = sum(p.numel() for p in model.parameters())
fp16_gb      = total_params * 2 / 1e9
fp32_gb      = total_params * 4 / 1e9

print(f"Model          : {MODEL_ID}")
print(f"Load time       : {load_time:.2f}s")
print(f"Parameters      : {total_params:,}")
print(f"FP16 RAM est    : {fp16_gb:.2f} GB")
print(f"FP32 RAM est    : {fp32_gb:.2f} GB")
print(f"Device map     : {model.hf_device_map}")

# Check which files were downloaded
cache_dir = os.path.expanduser("~/cache/huggingface/hub")
for root, dirs, files in os.walk(cache_dir):
    for f in files:
        if MODEL_ID.replace("/", "--") in root and f.endswith(".safetensors"):
            fpath = os.path.join(root, f)
            print(f" {f} ({os.path.getsize(fpath)/1e6:.0f} MB)")

```

GGUF File Internals

A GGUF file starts with a 4-byte magic number (*GGUF*), followed by a version uint32, then key-value metadata pairs encoding architecture, context length, embedding dimensions, tokeniser vocabulary, and quantisation parameters. Weight tensors follow, each with a name, shape, quantisation type, and packed quantised bytes. The format is self-describing — any GGUF-compatible runtime can load any GGUF file

without needing a separate config.json.

```
# Install llama-cpp-python to inspect GGUF files from Python
# pip install llama-cpp-python

from llama_cpp import Llama
import time

GGUF_PATH = "./phi-3-mini-4k-instruct-q4.gguf" # download from HF Hub

t0 = time.perf_counter()
model = Llama(
    model_path=GGUF_PATH,
    n_ctx=2048,          # context window size
    n_gpu_layers=0,      # 0 = CPU only; -1 = all layers on GPU
    verbose=False,
)
load_time = time.perf_counter() - t0

print(f"Load time      : {load_time:.2f}s")
print(f"Context length: {model.n_ctx()}")
print(f"Embed dim       : {model.n_embd()}")
print(f"Vocab size      : {model.n_vocab()}")

# Run inference
output = model("The key to fast inference is", max_tokens=40, echo=True)
print(output["choices"][0]["text"])

# Shell: inspect GGUF metadata without loading into Python
# ./llama-gguf-hash --xxh64 model.gguf          # verify checksum
# ./llama-cli -m model.gguf --info              # dump all metadata
```

Convert HuggingFace → GGUF Step by Step

```
#!/usr/bin/env bash
# Full conversion pipeline: SafeTensors HF model -> quantised GGUF

# 1. Install dependencies
pip install huggingface-hub transformers sentencepiece
git clone https://github.com/gggerganov/llama.cpp && cd llama.cpp
pip install -r requirements.txt
make -j$(nproc)          # compile llama-quantize and llama-cli binaries

# 2. Download model weights (SafeTensors format)
huggingface-cli download microsoft/Phi-3-mini-4k-instruct \
    --local-dir ./phi3-hf \
    --ignore-patterns "**.bin" "*.pt" # SafeTensors only

# 3. Convert to GGUF FP16 (lossless – use as quantisation source)
python convert_hf_to_gguf.py ./phi3-hf \
    --outfile ./phi3-f16.gguf \
    --outtype f16

# 4. Quantise to multiple levels for comparison
./llama-quantize phi3-f16.gguf phi3-Q4_0.gguf Q4_0
./llama-quantize phi3-f16.gguf phi3-Q4_K_M.gguf Q4_K_M
```

```
./llama-quantize phi3-f16.gguf phi3-Q8_0.gguf Q8_0
```

```
# 5. Compare sizes
```

```
echo "=== File sizes ==="
```

```
ls -lh phi3-*.gguf
```

```
# 6. Quick perplexity check (lower = better quality)
```

```
./llama-perplexity -m phi3-Q4_0.gguf -f wikitext-2-raw/wiki.test.raw -c 512
```

```
./llama-perplexity -m phi3-Q4_K_M.gguf -f wikitext-2-raw/wiki.test.raw -c 512
```

```
./llama-perplexity -m phi3-Q8_0.gguf -f wikitext-2-raw/wiki.test.raw -c 512
```

PART B — VRAM Calculation & Hardware Selection

The VRAM Formula

Before deploying any model the first question is: will it fit? The total GPU VRAM required at inference time is the sum of three components:

Total VRAM = Model Weights + KV Cache + Activation Overhead

Model Weights = num_parameters × bytes_per_parameter
= params × dtype_bytes × (1 + overhead_factor)
overhead_factor ≈ 0.1 (buffers, embedding tables, etc.)

KV Cache = 2 × num_layers × num_heads × head_dim × seq_len × batch_size × dtype_bytes
(the 2 is for Key and Value tensors)

Activations ≈ 10-20% of model weight size during inference (rule of thumb)

Practical simplified formula (inference only, no gradients):
VRAM_GB ≈ (params_B × dtype_bytes × 1.2) + kv_cache_GB

Example: LLaMA-3 8B in FP16 with 4096 context, batch=1
Weights = 8e9 × 2 bytes × 1.2 = 19.2 GB <-- won't fit on 16 GB GPU!
Weights = 8e9 × 2 bytes × 1.2 = 19.2 GB (FP16)
Weights = 8e9 × 0.5 bytes × 1.2 = 4.8 GB (INT4/AWQ) <-- fits on 8 GB GPU

Dtype Memory Footprint Reference

Data Type	Bytes/Param	7B Model	13B Model	70B Model	Notes
FP32	4	28 GB	52 GB	280 GB	Training default, rare at inference
BF16	2	14 GB	26 GB	140 GB	Training on modern GPUs (A100, H100)
FP16	2	14 GB	26 GB	140 GB	Inference default — good quality
INT8	1	7 GB	13 GB	70 GB	bitsandbytes LLM.int8() — minor quality loss
INT4 (GPTQ)	0.5	~4.5 GB	~8 GB	~40 GB	Good quality, GPU only
INT4 (AWQ)	0.5	~4.0 GB	~7.5 GB	~38 GB	Best INT4 quality, GPU only
GGUF Q4_K_M	~0.56	~4.1 GB	~7.9 GB	~41 GB	CPU/GPU — llama.cpp default

VRAM Calculator Script

```
def vram_estimate(
    params_billions: float,
    dtype: str = "fp16",
    context_len: int = 4096,
    batch_size: int = 1,
    num_layers: int = None,
    num_heads: int = None,
    head_dim: int = 128,
) -> dict:
    # Estimate VRAM requirement for LLM inference.
```


NVIDIA RTX 3060	12 GB	Consumer	7B INT4, 3B FP16
NVIDIA RTX 3090	24 GB	Consumer	13B FP16, 7B FP16 + headroom
NVIDIA RTX 4090	24 GB	Consumer	13B FP16, 70B INT4 (multi-GPU)
NVIDIA A10G	24 GB	Cloud (g5)	13B FP16, 7B FP16 easily
NVIDIA A100 40G	40 GB	Data centre	30B FP16, 70B INT4
NVIDIA A100 80G	80 GB	Data centre	70B FP16, 405B INT4 (multi-GPU)
NVIDIA H100 80G	80 GB	Data centre	70B FP16 + large KV cache
Apple M2 Ultra	192 GB	Workstation	70B FP16 (unified memory)

Rule of thumb: 7B FP16 = ~14 GB VRAM. 7B INT4 = ~4–5 GB VRAM. Always add 20% headroom for KV cache and activations. When in doubt, use the VRAM calculator script above.

PART C — CPU vs GPU Trade-offs

Why GPUs Are So Much Faster

LLM inference is dominated by **matrix-vector multiplications** — multiplying the current token's embedding vector against each weight matrix. A modern GPU (e.g. A100) has ~312 TFLOPS of FP16 throughput and ~2 TB/s of memory bandwidth. A modern CPU (e.g. Xeon 8380) delivers ~1 TFLOPS and ~200 GB/s. The gap is 100-300x in raw compute. However, token generation is largely **memory-bandwidth bound** (not compute bound) because each token only requires reading the full weight matrix once. The practical speedup is therefore 10-50x, not 300x.

Dimension	CPU (e.g. Xeon / Apple M2)	GPU (e.g. A100 / RTX 4090)
FP16 TFLOPS	~1–4 TFLOPS	~100–312 TFLOPS
Memory bandwidth	~50–200 GB/s	~900 GB/s – 2 TB/s
VRAM / RAM	128–512 GB system RAM	8–80 GB VRAM (fast)
Token gen speed (7B)	3–15 tok/s	40–100+ tok/s
TTFT (4096 ctx)	5–30 s (prompt processing)	0.5–2 s
Power draw	65–350 W	150–700 W
Cost (cloud/hr)	\$0.05–0.20/hr (CPU instance)	\$1–4/hr (A10G/A100)
Best for	Dev/test, low concurrency, GGUF	Production, high throughput

CPU Strengths

- **Unified memory:** Apple Silicon (M1/M2/M3) shares RAM between CPU and GPU cores, allowing 70B models to run on a MacBook Pro with 192 GB unified memory — impossible on discrete GPUs without expensive multi-GPU setups.
- **Large context windows:** GGUF on CPU can handle 128K context without VRAM OOM. GPUs are VRAM-constrained on long contexts.
- **Cost:** CPU inference costs 20-40x less per hour on cloud. Viable for low-QPS workloads.
- **Simplicity:** No CUDA drivers, no GPU scheduling, no OOM crashes.

Practical Benchmark Script — Same Model, CPU vs GPU

```
import time, torch
from transformers import AutoTokenizer, AutoModelForCausalLM

MODEL_ID = "gpt2"      # swap to a 7B model when GPU is available
PROMPT  = "The fundamental challenge of distributed machine learning is"
NEW_TOKS = 100

def benchmark(device_label, device, dtype):
    tok = AutoTokenizer.from_pretrained(MODEL_ID)
    model = AutoModelForCausalLM.from_pretrained(
        MODEL_ID, torch_dtype=dtype
```

```

).to(device)
model.eval()

inputs = tok(PROMPT, return_tensors="pt").to(device)

# Warm-up run (fills caches, JIT compiles CUDA kernels)
with torch.no_grad():
    model.generate(**inputs, max_new_tokens=10,
                  pad_token_id=tok.eos_token_id)

# Timed run
if device != "cpu":
    torch.cuda.synchronize()
    t0 = time.perf_counter()
    with torch.no_grad():
        out = model.generate(**inputs, max_new_tokens=NEW_TOKS,
                            do_sample=False,
                            pad_token_id=tok.eos_token_id)
    if device != "cpu":
        torch.cuda.synchronize()
    elapsed = time.perf_counter() - t0

new_toks = out.shape[1] - inputs["input_ids"].shape[1]
tps      = new_toks / elapsed

if device != "cpu":
    mem_mb = torch.cuda.max_memory_allocated() / 1e6
else:
    mem_mb = 0    # use tracemalloc for CPU

print(f" {device_label:<18} {new_toks:>6} tok {elapsed:>7.2f}s {tps:>7.1f} tok/s {mem_mb:>7.0f} MB")

print(f" {'Device':<18} {'Tokens':>6} {'Time':>7} {'Speed':>11} {'VRAM/RAM':>9}")
print(" " + "-" * 62)

benchmark("CPU (FP32)", "cpu", torch.float32)
benchmark("CPU (FP16)", "cpu", torch.float16)

if torch.cuda.is_available():
    benchmark("GPU (FP32)", "cuda", torch.float32)
    benchmark("GPU (FP16)", "cuda", torch.float16)
else:
    print(" GPU not available – run on a CUDA-enabled machine for GPU rows")

```

GGUF CPU Benchmark with llama.cpp

```

#!/usr/bin/env bash
# Benchmark llama.cpp (GGUF) on CPU – uses AVX-512 / NEON SIMD kernels

MODEL="./phi3-Q4_K_M.gguf"
PROMPT="The fundamental challenge of distributed systems is"

# Benchmark with different thread counts
for threads in 1 2 4 8; do
    echo "=== Threads: $threads ==="
    ./llama-bench \

```

```

-m $MODEL \
-t $threads \
-n 100 \
-p 64 \
2>&1 | grep -E "pp|tg"
done

# Full verbose run (shows all timing fields)
./llama-cli \
-m $MODEL \
-p "$PROMPT" \
-n 80 \
--threads 8 \
--verbose-prompt \
2>&1 | tail -20

```

When to Use CPU vs GPU — Decision Matrix

Scenario	CPU	GPU	Reasoning
Development & local testing	YES	OK	No GPU needed for workflow testing
Production, >10 concurrent users	NO	YES	GPU throughput needed for latency SLA
70B+ model on a single machine	YES*	Multi	*Apple Silicon unified memory only
Long context (>32K tokens)	OK	YES	CPU handles large RAM; GPU VRAM limited
Cost-optimised, <5 QPS	YES	NO	CPU cloud is 20-40x cheaper/hr
Batch inference / offline processing	NO	YES	GPU continuous batching crucial
Fine-tuning (LoRA / full)	NO	YES	Backprop requires GPU compute & VRAM

Testing & Metrics

Verify GGUF Conversion Quality (Perplexity)

```

# Download WikiText-2 test set for perplexity evaluation
wget https://raw.githubusercontent.com/salesforce/awd-lstm-lm/master/data/wikitext-2/wiki.test.tokens

# Compute perplexity for each quantisation level
for quant in f16 Q8_0 Q4_K_M Q4_0 Q3_K_M; do
  echo -n "phi3-${quant}.gguf PPL: "
  ./llama-perplexity \
    -m ./phi3-${quant}.gguf \
    -f wiki.test.tokens \
    -c 512 --chunks 10 \
    2>&1 | grep "Final estimate"
done

# Expected output (lower perplexity = better quality):
# phi3-f16.gguf      PPL: ~5.8 (reference)
# phi3-Q8_0.gguf    PPL: ~5.8 (negligible loss)
# phi3-Q4_K_M.gguf  PPL: ~6.0 (<4% loss – acceptable)

```

```
# phi3-Q4_0.gguf      PPL: ~6.2  (~7% loss)
# phi3-Q3_K_M.gguf    PPL: ~6.8  (~17% loss – noticeable)
```

Metrics to Record for Each Format/Hardware Combo

Metric	How to Measure	Target
File size (GB)	ls -lh *.gguf	4–5 GB for 7B Q4_K_M
Load time (s)	time ollama run model " 2>&1	< 10s cold, < 0.1s warm
VRAM used (MB)	nvidia-smi / torch.cuda.max_memory	< GPU VRAM budget
Perplexity (PPL)	llama-perplexity on WikiText-2	Within 5% of FP16 baseline
Generation speed	eval_count / eval_duration (Ollama)	>10 tok/s CPU, >40 GPU
TTFT (s)	Time to first token in stream	< 3s CPU, < 1s GPU

Key Takeaways

- **SafeTensors** uses mmap zero-copy loading — fast cold start and no pickle RCE risk. Always prefer it over .bin for new work.
- **GGUF is self-describing** — architecture, tokeniser, and quantised weights in one file. No config.json or tokeniser.json needed separately.
- **VRAM formula:** params_B x dtype_bytes x 1.2 (overhead) + KV cache. Use the calculator script before provisioning hardware.
- **Q4_K_M is the GGUF sweet spot** — perplexity within ~4% of FP16 at 30% the size.
- **GPU is memory-bandwidth bound during generation**, not compute bound — INT4 quantisation helps because it reduces the bytes read per token, not just VRAM usage.
- **CPU is viable** for dev/test and low-QPS workloads, especially on Apple Silicon where unified memory lets you run 70B models impossible on consumer GPUs.

Next topic: **Day 4 — Quantization:** AWQ INT4 with autoawq, perplexity comparison before and after quantisation, VRAM savings measurement, and KV cache concepts.